

PROJECT REPORT



Project Name:

Memory Management Simulator

Session:2023-2027

(Reg. No. 2023-UOK-03674, 2023-UOK-03679, 2023-UOK-03683)

Submitted To: Mr. Mehtab Mushtaq

Subject: Operating System

Submitted By:

Siraj ud din (74), Farhan Fiaz (79), Owais Shezad (83)

Date of Submission: February13, 2026

**DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF COMPUTING AND ENGINEERING
UNIVERSITY OF KOTLI AZAD JAMMU AND KASHMIR**

Memory Management Simulator

(Reg. No. 2023-UOK-03674, 2023-UOK-03679, 2023-UOK-03683)

Submitted To: Mr. Mehtab Mushtaq

Subject: Operating System

Session:2023-2027

Submitted By:

Siraj ud din (74), Farhan Fiaz (79), Owais Shezad (83)

Date of Submission: February13, 2026

**DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF COMPUTING AND ENGINEERING
UNIVERSITY OF KOTLI AZAD JAMMU AND KASHMIR**

Table of Contents

- 1. REQUIREMENT SPECIFICATION 1
- 1.1. FUNCTIONAL REQUIREMENTS 1
- 1.2. NO- FUNCTIONAL REQUIREMENTS 1
- 2. SOFTWARE ARCHITECTURE DIAGRAM 2
- 2.1-Architecture Diagram-User Interface & Memory Manager 2
- Explanation:** 2
- 2.2-Allocation Algorithm 3
- Explanation:** 3
- 3 Component Diagram..... 4
- Explanation:** 4

Table of Figures

Figure 1 User Interface and Memory Manager.....2
Figure 2 Allocation Algorithms3
Figure 3 Complete Component diagram4

1. REQUIREMENT SPECIFICATION

This section describes the functional and non-functional requirements of the **Memory Management Simulator**. These requirements define what the system should do and the constraints under which it should operate. The architecture of the system is designed according to these requirements.

1.1. FUNCTIONAL REQUIREMENTS

- ◆ Allow users to input multiple processes and their memory requirements.
- ◆ Implement memory allocation algorithms: First Fit, Next Fit, Best Fit, and Worst Fit.
- ◆ Simulate paging and segmentation, showing memory as frames or logical segments.
- ◆ Allow memory deallocation when a process terminates or no longer needs memory.

1.2. NON- FUNCTIONAL REQUIREMENTS

- ◆ User-friendly and intuitive interface for easy navigation.
- ◆ Compatible with Windows and Linux platforms.
- ◆ Reliable and accurate visualization of memory management processes.
- ◆ Lightweight and implemented using open-source tools (Python and Tkinter).

2. SOFTWARE ARCHITECTURE DIAGRAM

The Memory Management Simulator is a software project designed to simulate the allocation, deallocation, and organization of memory for multiple processes, using various memory management techniques such as paging, segmentation, and different allocation strategies (First Fit, Best Fit, Worst Fit). The simulator allows users to interact with memory operations through a graphical interface, while the Memory Manager coordinates memory allocation, interacts with algorithms, and maintains memory and process records via dedicated components. The component diagram illustrates the architecture of the system, showing the modular structure, the relationships between components, and how services are provided and required across layers. It highlights the separation of concerns, loose coupling, and flexibility of the system, enabling easy maintenance, extension, and understanding of the memory management workflow.

2.1-Architecture Diagram-User Interface & Memory Manager

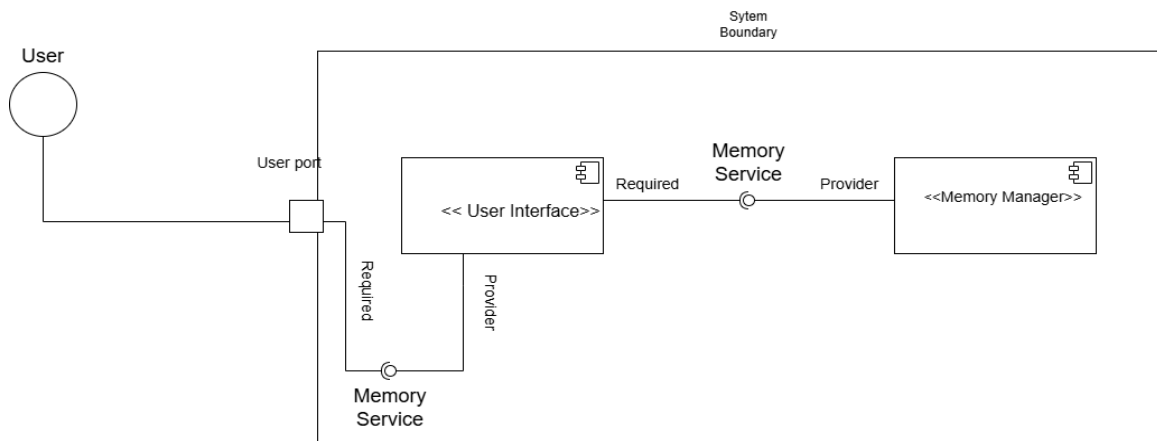


Figure 1 User Interface and Memory Manager

Explanation:

Initially, the system architecture consisted of two primary components: the Graphical User Interface (GUI) and the Memory Manager. The GUI serves as the presentation layer of the system and is responsible for handling user interaction, such as accepting memory allocation and deallocation requests and displaying the current memory status. It communicates with the Memory Manager through a defined service interface. The Memory Manager acts as the core controlling component of the system, receiving requests from the GUI and coordinating all memory-related operations. It processes allocation and deallocation requests and ensures proper management of memory resources within the simulator. The separation between GUI and Memory Manager ensures modularity and clear division of responsibilities.

2.2-Allocation Algorithm

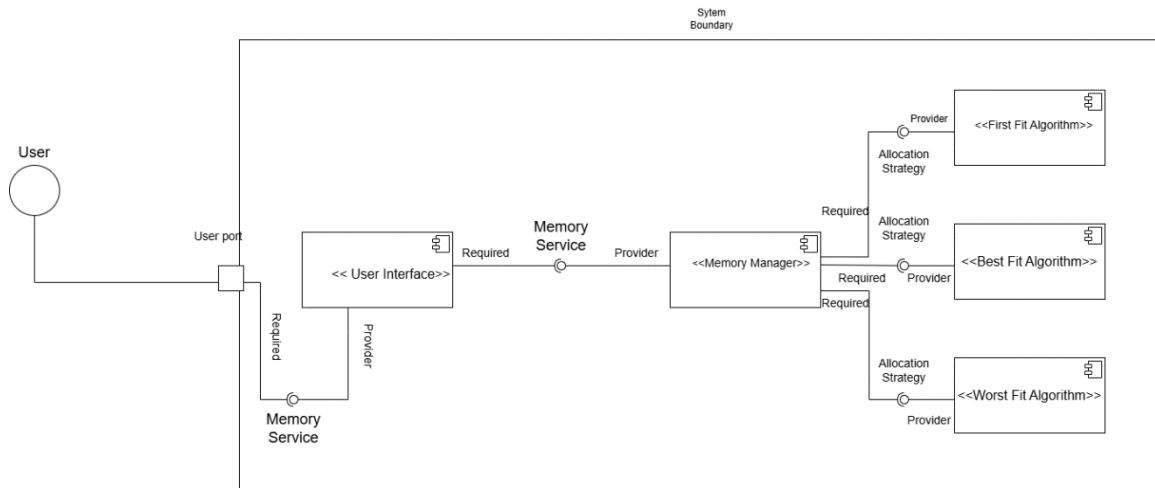


Figure 2 Allocation Algorithms

Explanation:

After establishing the basic structure, three allocation strategy components — First Fit, Best Fit, and Worst Fit — were added to the system. These components implement different memory allocation algorithms and provide a common allocation interface required by the Memory Manager. The Memory Manager depends on this interface and can dynamically utilize any of these strategies without changing its internal logic. The First Fit algorithm selects the first suitable memory block, the Best Fit algorithm selects the smallest suitable block to reduce fragmentation, and the Worst Fit algorithm selects the largest available block. The inclusion of these components enhances flexibility, promotes reusability, and follows a strategy-based architectural approach.

3. Component Diagram

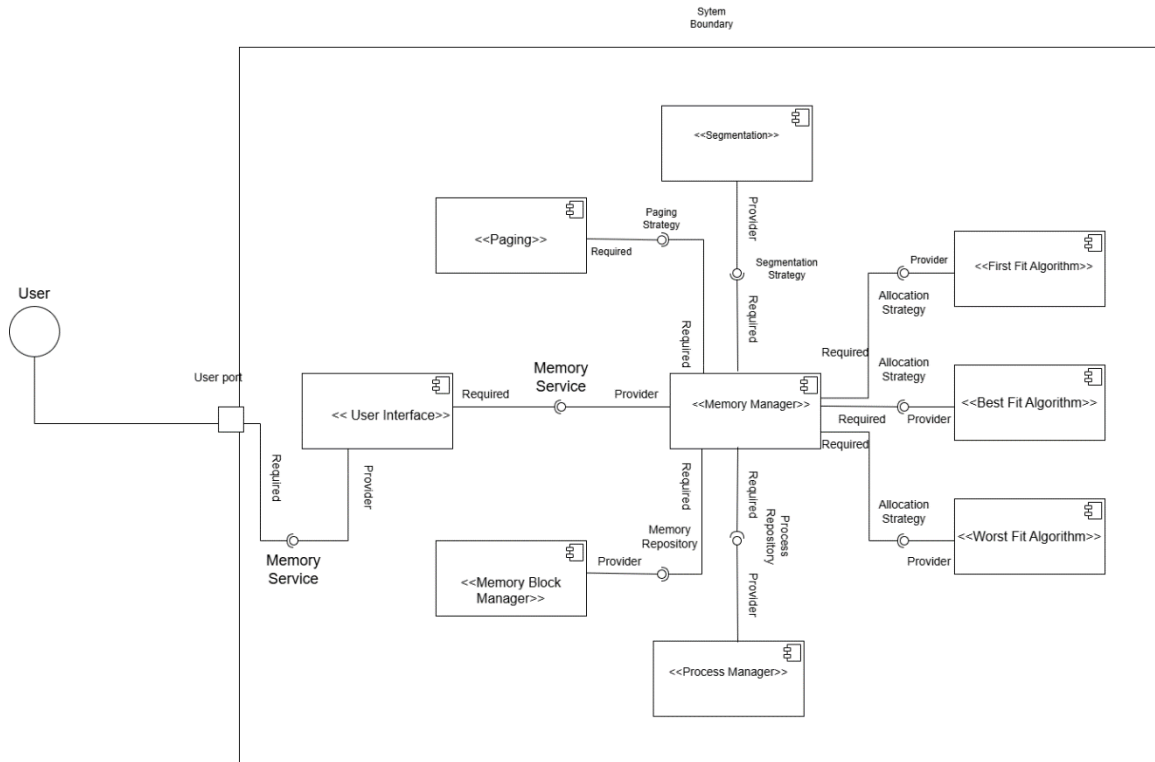


Figure 3 – Complete Component diagram

Explanation:

In the final stage, four additional components were integrated: Paging, Segmentation, Memory Block Manager, and their coordination with the Memory Manager. The Paging component manages memory using fixed-size pages and frames, reducing external fragmentation, while the Segmentation component organizes memory into logical variable-sized segments reflecting program structure. Both components provide memory organization mechanisms that are controlled by the Memory Manager. The Memory Block Manager is responsible for maintaining records of free and allocated memory blocks and acts as a data management component. The Memory Manager interacts with the Paging and Segmentation components to apply the selected memory technique and communicates with the Memory Block Manager to update and retrieve memory status information. This structured relationship ensures proper separation of control logic, memory organization mechanisms, and memory data handling, resulting in a layered and enterprise-level architectural design.

